UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Programa educativo: Ingeniería en Tecnologías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Actividad parcial 2

Tema: Algoritmos de búsqueda local: Hill Climbing y Simulated Annealing

Alumno: José Manuel Arizmendi Varela

Grupo: 172

Cuatrimestre: Séptimo

Docente: Mtro. Javier García Lara

1. INTRODUCCIÓN

La inteligencia artificial utiliza diferentes algoritmos para encontrar soluciones a problemas de optimización. Estos problemas consisten en buscar una alternativa que permita obtener el mejor resultado posible, por ejemplo, reducir costos, mejorar una ruta o aprovechar mejor los recursos disponibles.

Una de las técnicas utilizadas para resolver estos problemas es la búsqueda local. Este tipo de búsqueda trabaja con una solución inicial y explora alternativas cercanas para encontrar una solución mejor.

Entre los algoritmos de búsqueda local se encuentran Hill Climbing y Simulated Annealing. Ambos permiten explorar soluciones y evaluar sus resultados, pero utilizan estrategias diferentes.

Hill Climbing selecciona movimientos que mejoran la solución actual. Aunque es un algoritmo sencillo, puede quedarse atrapado en un mínimo local y no encontrar la mejor solución global.

Por otra parte, Simulated Annealing permite aceptar temporalmente algunas soluciones peores. Esta característica le da la posibilidad de explorar otras regiones del espacio de búsqueda y evitar, en determinadas circunstancias, quedar atrapado en un mínimo local.

En este trabajo se implementan ambos algoritmos en Python para minimizar una función matemática con diferentes regiones de interés. Se establecerán los parámetros de búsqueda, se generarán gráficas de convergencia y se compararán los resultados obtenidos.

2. OBJETIVOS

2.1. Objetivo general

Implementar y comparar los algoritmos Hill Climbing y Simulated Annealing mediante Python para resolver un problema de optimización, analizando su funcionamiento, sus parámetros y la calidad de las soluciones encontradas.

2.2. Objetivos específicos

- Comprender el funcionamiento de los algoritmos de búsqueda local.
- Definir una función objetivo que permita evaluar diferentes soluciones.
- Establecer el espacio de búsqueda y la vecindad de las soluciones.
- Implementar Hill Climbing mediante movimientos que mejoren el resultado actual.
- Implementar Simulated Annealing utilizando una temperatura inicial y un proceso de enfriamiento.
- Generar gráficas para observar la evolución de los algoritmos.
- Comparar las soluciones obtenidas y explicar las diferencias entre ambos métodos.

3. PLANTEAMIENTO DEL PROBLEMA

El problema consiste en encontrar el valor de una variable que permita minimizar una función matemática dentro de un intervalo definido.

Este tipo de problema representa una situación general de optimización. En una aplicación real, la función objetivo podría representar costos de transporte, consumo de recursos o gastos de producción.

Para esta actividad se utilizará una función matemática diseñada para tener diferentes regiones de búsqueda. De esta manera, será posible observar cómo se comportan los algoritmos cuando existen varias soluciones locales.

El propósito no es encontrar una ruta física ni calcular un costo empresarial real, sino estudiar el comportamiento de dos algoritmos de optimización mediante un ejemplo controlado.

3.1. Definición del problema

Estado inicial: una solución representada por un valor numérico de la variable x.

Espacio de búsqueda: todos los valores de x comprendidos entre -2 y 2.

Acciones: modificar ligeramente el valor actual de x para generar una solución vecina.

Función objetivo: evaluar el costo asociado con cada valor de x.

Meta: encontrar el valor de x que produzca el menor costo posible.

Restricciones: los valores de x deben permanecer dentro del intervalo [-2, 2].

Criterio de evaluación: comparar el costo de las soluciones y registrar la mejor alternativa encontrada por cada algoritmo.

4. FUNCIÓN OBJETIVO

La función utilizada en este proyecto es:

$f(x) = (x^2 - 1)^2 + 0.15x$

El objetivo es minimizar el valor de f(x) dentro del intervalo:

$-2 \leq x \leq 2$

4.1. Explicación de la función

La función combina un término cuadrático elevado al cuadrado con un término lineal.

El primer término, $(x^2 - 1)^2$, produce dos regiones de interés alrededor de x = -1 y x = 1.

El segundo término, 0.15x, modifica ligeramente los valores de la función y hace que uno de los mínimos sea mejor que el otro.

Esta característica permite estudiar el problema de los mínimos locales.

En este caso, el mínimo global se encuentra aproximadamente en:

$x \approx -1.0187$

El valor correspondiente de la función es aproximadamente:

$f(x) \approx -0.1514$

Estos valores son referencias matemáticas para comparar los resultados de los algoritmos. La solución que encuentre cada programa puede presentar pequeñas diferencias debido a los movimientos utilizados durante la búsqueda.

4.2. Importancia de la función objetivo

La función objetivo permite asignar un valor numérico a cada solución candidata.

Cuando se busca minimizar una función, una solución con un valor menor es preferible a otra con un valor mayor.

Por ejemplo, si una solución tiene un costo de 0.5 y otra tiene un costo de -0.1, la segunda es mejor para este problema porque su valor es menor.

Este criterio será utilizado por ambos algoritmos para decidir qué soluciones conviene conservar.

5. ALGORITMO HILL CLIMBING

5.1. Descripción

Hill Climbing, también conocido como búsqueda de ascenso de colina, es un algoritmo de búsqueda local que intenta mejorar progresivamente una solución.

Aunque su nombre hace referencia a subir una montaña, también puede utilizarse para minimizar funciones. En este caso, el algoritmo busca descender hacia valores cada vez menores de la función objetivo.

El procedimiento comienza con una solución inicial y genera una solución vecina. Si la nueva solución tiene un costo menor, se acepta. Si tiene un costo mayor o igual, se conserva la solución actual.

El proceso se repite hasta completar el número de iteraciones establecido.

5.2. Funcionamiento

El algoritmo realiza las siguientes acciones:

1. Selecciona una solución inicial.
2. Genera un vecino cercano.
3. Calcula el costo de la nueva solución.
4. Compara el costo nuevo con el costo actual.
5. Acepta el vecino si mejora la solución.
6. Repite el proceso hasta completar las iteraciones.
7. Devuelve la mejor solución encontrada.

5.3. Ventajas y limitaciones

Una ventaja de Hill Climbing es que resulta sencillo de implementar y permite encontrar soluciones buenas sin explorar todo el espacio de búsqueda.

Sin embargo, presenta una limitación importante: solamente acepta movimientos que mejoran la solución actual.

Si llega a un mínimo local, puede dejar de avanzar aunque exista otro mínimo con un costo inferior en otra región del espacio de búsqueda.

Esta limitación se analizará al comparar los resultados con Simulated Annealing.

6. ALGORITMO SIMULATED ANNEALING

6.1. Descripción

Simulated Annealing, también conocido como recocido simulado, es un algoritmo de optimización inspirado en el proceso físico de enfriamiento controlado de ciertos materiales.

Su principal característica es que puede aceptar movimientos que empeoran temporalmente la solución actual.

Esto permite explorar otras regiones del espacio de búsqueda y aumenta la posibilidad de escapar de un mínimo local.

La probabilidad de aceptar una solución peor depende de la diferencia de costo y de la temperatura actual del algoritmo.

6.2. Funcionamiento

El algoritmo realiza las siguientes acciones:

1. Selecciona una solución inicial.
2. Establece una temperatura inicial.
3. Genera una solución vecina.
4. Calcula el cambio de costo entre ambas soluciones.
5. Acepta siempre una mejora.
6. Si la solución empeora, calcula la probabilidad de aceptarla.
7. Reduce gradualmente la temperatura.
8. Conserva el registro de la mejor solución encontrada.
9. Repite el proceso hasta completar las iteraciones.

6.3. Temperatura y enfriamiento

La temperatura controla la disposición del algoritmo para aceptar soluciones peores.

Cuando la temperatura es alta, existe una mayor posibilidad de aceptar movimientos que aumentan el costo. Esto permite explorar diferentes regiones.

Cuando la temperatura disminuye, el algoritmo se vuelve más selectivo y tiende a conservar soluciones que presentan mejores resultados.

El enfriamiento debe realizarse de forma gradual. Si la temperatura disminuye demasiado rápido, el algoritmo podría perder la posibilidad de explorar otras regiones antes de encontrar una buena solución.

6.4. Ventajas y limitaciones

Una ventaja de Simulated Annealing es que puede escapar de algunos mínimos locales gracias a la aceptación probabilística de soluciones peores.

Sin embargo, su resultado depende de parámetros como la temperatura inicial, el enfriamiento, el tamaño de los movimientos y el número de iteraciones.

Por esta razón, no garantiza encontrar el mínimo global en todas las ejecuciones, aunque puede ofrecer mejores resultados que una búsqueda puramente voraz en determinados problemas.

7. PARÁMETROS UTILIZADOS

Para comparar los algoritmos se utilizarán los siguientes parámetros:

Parámetro	**Hill Climbing**	**Simulated Annealing**
Solución inicial	x = 0.8	x = 0.8
Intervalo de búsqueda	[-2, 2]	[-2, 2]
Número de iteraciones	2000	3000
Tamaño del movimiento	0.02	Desviación estándar de 0.35
Temperatura inicial	No aplica	2.0
Factor de enfriamiento	No aplica	0.998
Criterio de aceptación	Acepta solamente mejoras	Acepta mejoras y algunas soluciones peores
Objetivo	Minimizar f(x)	Minimizar f(x)

Para Hill Climbing, los movimientos se realizan mediante pequeños cambios de -0.02 o +0.02.

Para Simulated Annealing, los movimientos se generan mediante una distribución normal centrada en la solución actual, utilizando una desviación estándar de 0.35.

El factor de enfriamiento es 0.998, lo que significa que la temperatura se multiplica por ese valor después de cada iteración.

Ambos algoritmos parten del mismo valor inicial, pero utilizan diferentes estrategias para explorar el espacio de búsqueda.

8. PROCEDIMIENTO EXPERIMENTAL

El experimento se realizará mediante Python utilizando las bibliotecas Matplotlib y las herramientas incluidas en el propio lenguaje.

Primero se define la función objetivo y se establecen los límites del espacio de búsqueda.

Después se implementa Hill Climbing, que genera pequeños movimientos y acepta únicamente aquellos que disminuyen el costo.

Posteriormente se implementa Simulated Annealing, que utiliza una temperatura para controlar la aceptación de movimientos que pueden empeorar temporalmente la solución.

Durante cada ejecución se registra la mejor solución encontrada hasta el momento. Esta información permite generar una gráfica de convergencia.

La comparación se realiza considerando el valor final de x, el costo obtenido y la evolución de la mejor solución durante las iteraciones.

Para facilitar la reproducción del experimento, el programa utiliza semillas aleatorias fijas. Esto permite repetir las mismas secuencias de números aleatorios cuando se ejecuta con el mismo código y un entorno equivalente.

9. RESULTADOS Y COMPARACIÓN

Con los parámetros definidos en el programa y las semillas aleatorias correspondientes, se espera que Hill Climbing encuentre una solución cercana al mínimo local positivo, mientras que Simulated Annealing puede explorar la región negativa y aproximarse al mínimo global.

Los valores de referencia obtenidos con esta configuración son los siguientes:

Criterio	**Hill Climbing**	**Simulated Annealing**
Valor inicial de x	0.8	0.8
Valor final aproximado de x	0.9800	-1.0175
Costo final aproximado	0.1486	-0.1514
Iteraciones	2000	3000
Estrategia	Acepta mejoras	Acepta mejoras y algunas soluciones peores

Criterio	Hill Climbing	Simulated Annealing
Resultado esperado	Mínimo local	Aproximación al mínimo global

Los valores anteriores corresponden a la configuración experimental indicada. Si se modifican los parámetros, las semillas aleatorias o el código, los resultados pueden cambiar.

9.1. Interpretación de los resultados

Hill Climbing comienza en x = 0.8 y encuentra una región en la que los movimientos pequeños permiten reducir el costo. Sin embargo, al llegar a un mínimo local, deja de encontrar mejoras mediante los movimientos permitidos.

Por esta razón, el algoritmo puede terminar con una solución cercana a x = 0.98, cuyo costo es aproximadamente 0.1486.

Simulated Annealing comienza en el mismo punto, pero puede aceptar temporalmente movimientos que aumentan el costo. Esto le permite explorar otras regiones y encontrar una solución cercana a x = -1.0175, cuyo costo es aproximadamente -0.1514.

Como el objetivo es minimizar la función, la solución de Simulated Annealing es mejor en este experimento porque obtiene un costo menor.

Este resultado ilustra una ventaja potencial del recocido simulado: la posibilidad de escapar de un mínimo local.

Sin embargo, no significa que Simulated Annealing siempre sea mejor. Su desempeño depende de los parámetros utilizados y de la naturaleza del problema.

10. GRÁFICAS DE CONVERGENCIA

El programa genera tres gráficas principales.

10.1. Gráfica de la función objetivo

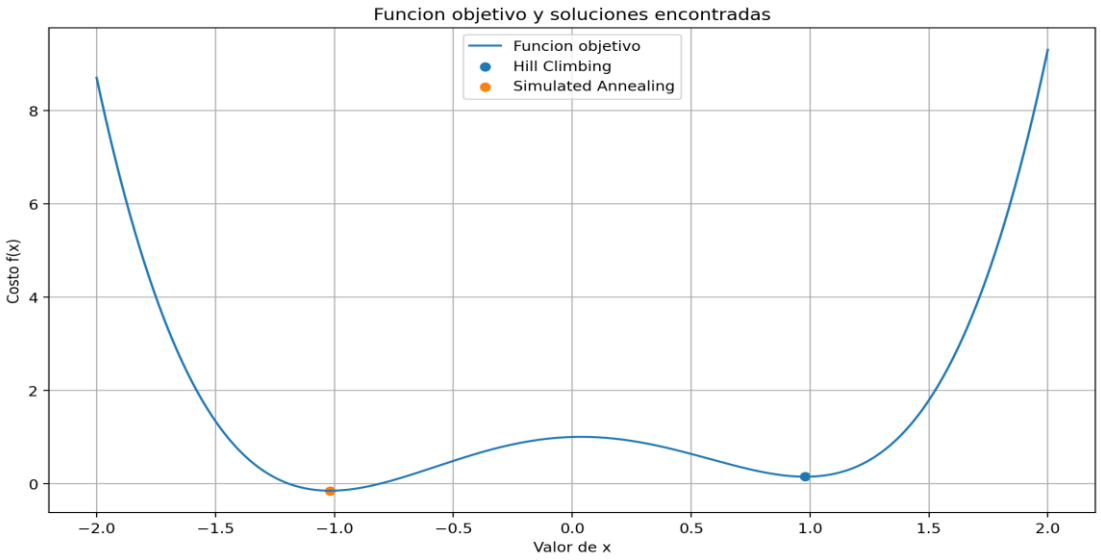


Figura 1. Función objetivo y soluciones encontradas por los algoritmos Hill Climbing y Simulated Annealing. Fuente: elaboración propia.

Esta gráfica muestra cómo cambia el valor de f(x) dentro del intervalo [-2, 2].

Permite identificar las regiones donde la función presenta valores bajos y comparar visualmente las soluciones encontradas.

10.2. Convergencia de Hill Climbing

La gráfica muestra la mejor solución encontrada por Hill Climbing durante las iteraciones.

Se espera observar una disminución del costo al principio y posteriormente una estabilización cuando el algoritmo deja de encontrar mejoras.

10.3. Convergencia de Simulated Annealing

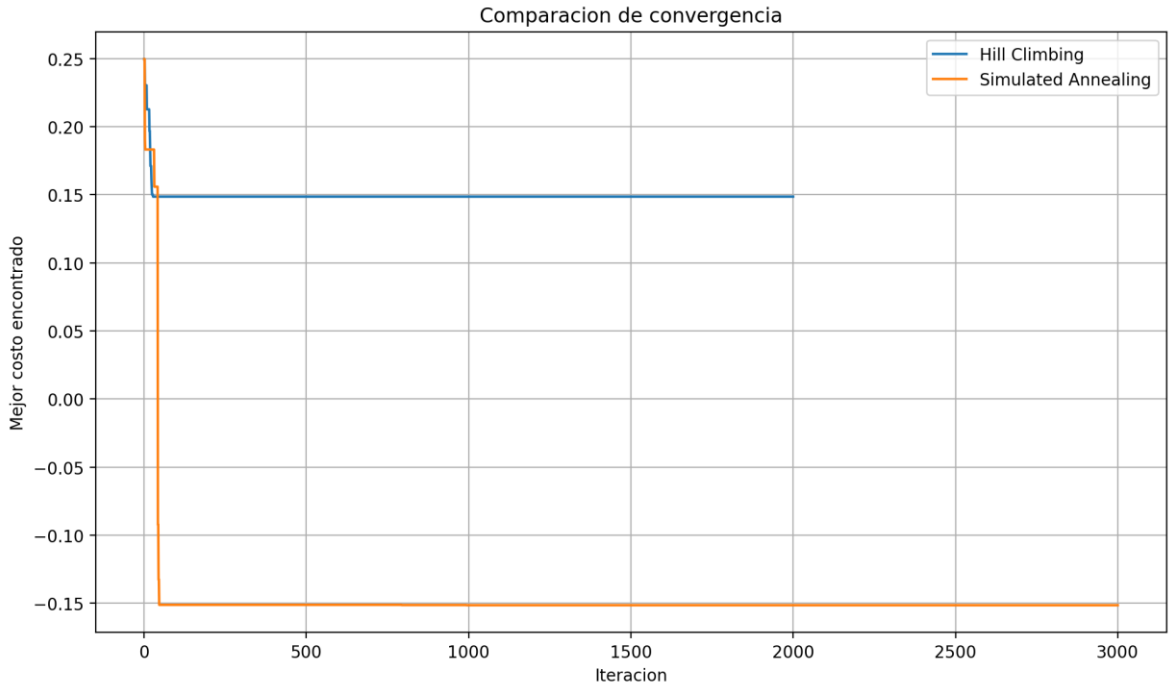


Figura 2. Comparación de la convergencia de los algoritmos Hill Climbing y Simulated Annealing. Fuente: elaboración propia.

La gráfica muestra la evolución de la mejor solución encontrada por Simulated Annealing.

En algunas etapas, el algoritmo puede aceptar movimientos peores en su solución actual. Sin embargo, la gráfica registra la mejor solución encontrada hasta ese momento, por lo que el mejor costo registrado no debe aumentar.

Al comparar las gráficas se puede analizar si alguno de los algoritmos encuentra una solución mejor y qué tan rápido mejora el resultado.

11. CONCLUSIONES

Durante esta actividad comprendí que los algoritmos de búsqueda local permiten resolver problemas de optimización mediante la exploración de diferentes soluciones.

Hill Climbing resultó útil porque utiliza una estrategia sencilla: generar una solución vecina y aceptarla cuando mejora el costo. Sin embargo, su principal limitación es que puede quedarse atrapado en un mínimo local y dejar de explorar otras regiones.

Por otra parte, Simulated Annealing utiliza una estrategia diferente porque permite aceptar algunas soluciones peores de manera probabilística. Esta característica le proporciona una oportunidad para escapar de mínimos locales y encontrar soluciones de menor costo.

Al comparar ambos algoritmos en la función seleccionada, se observa que Simulated Annealing puede encontrar una solución mejor que Hill Climbing. El resultado demuestra la importancia de elegir adecuadamente el algoritmo y configurar sus parámetros.

También comprendí que las gráficas de convergencia son útiles para observar cómo evoluciona la búsqueda y determinar si los algoritmos mejoran sus resultados durante las iteraciones.

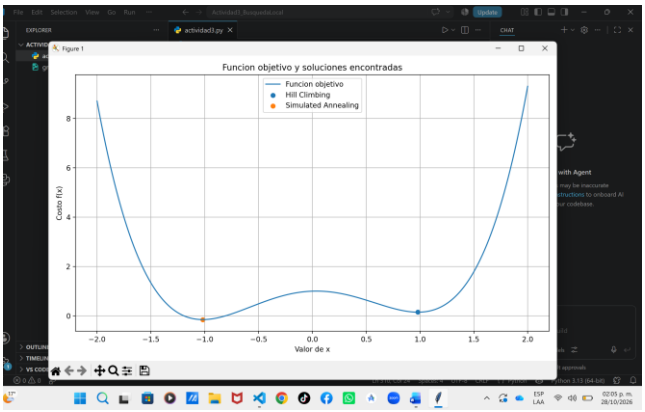
Finalmente, considero que estos métodos pueden aplicarse a problemas de rutas, distribución de recursos y planificación, siempre que se defina una función objetivo apropiada y se establezcan las restricciones del problema.

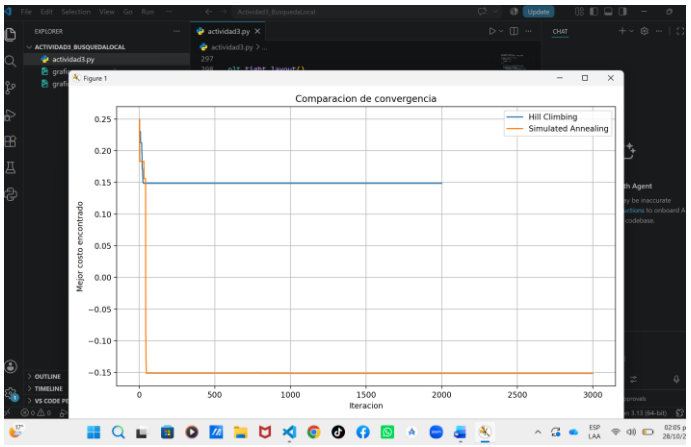
12. DECLARACIÓN SOBRE EL USO DE INTELIGENCIA ARTIFICIAL

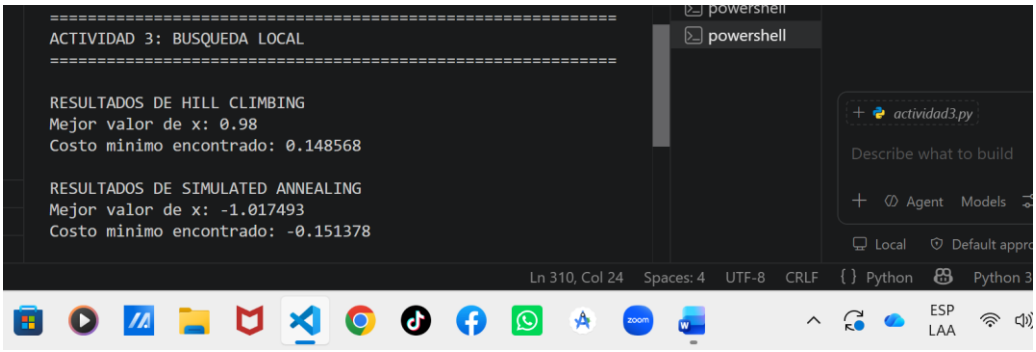
Se utilizaron herramientas de inteligencia artificial como apoyo para organizar el informe, comprender los conceptos de búsqueda local y preparar una primera versión del código.

El programa debe ejecutarse y verificarse antes de entregar el trabajo. El estudiante es responsable de revisar los resultados, comprender el funcionamiento de los algoritmos y comprobar que las conclusiones correspondan con el experimento realizado.

13. ANEXOS







13. REFERENCIAS BIBLIOGRÁFICAS

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

NetworkX Developers. (s. f.). *NetworkX documentation*. https://networkx.org/documentation/stable/

Python Software Foundation. (s. f.). *The Python tutorial*. https://docs.python.org/3/tutorial/

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.